

The "switch" Statement - Simple Hinglish Summary

Source: <https://javascript.info/switch>

Main baat

`switch` statement **kai** `if` **checks ki jagah** use ho sakta hai. Ek value ko **kai variants se compare** karne ka zyada saaf tarika hai.

1. Syntax

`switch` me ek ya zyada `case` blocks hote hain aur ek optional `default`.

```
javascript

switch(x) {
  case 'value1': // if (x === 'value1')
    ...
    [break]

  case 'value2': // if (x === 'value2')
    ...
    [break]

  default:
    ...
    [break]
}
```

Kaise kaam karta hai:

- `x` ki value ko pehle `case` se, phir dusre se, aise **strict equality** (`===`) se check kiya jata hai.
- Match milne par us `case` se code chalna shuru hota hai, **agle** `break` **tak** (ya `switch` ke end tak).
- Koi `case` match na ho to `default` wala code chalta hai (agar hai).

2. Example

```
javascript
```

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
    break;
  case 4:
    alert( 'Exactly!' );
    break;
  case 5:
    alert( 'Too big' );
    break;
  default:
    alert( "I don't know such values" );
}
```

`a = 4` hai, to pehle `3` se match fail, phir `4` se match hua. `case 4` se `break` tak code chala.
Output: **Exactly!**

`break` na ho to?

`break` na ho to execution agle `case` me bina check kiye chalta rehta hai.

javascript

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
  case 4:
    alert( 'Exactly!' );
  case 5:
    alert( 'Too big' );
  default:
    alert( "I don't know such values" );
}
```

Yahan teen alerts ek ke baad ek aayenge: `Exactly!`, `Too big`, `I don't know such values`.

Koi bhi expression chalta hai

`switch` aur `case` dono me **arbitrary expressions** likh sakte ho:

javascript

```
let a = "1";
let b = 0;

switch (+a) {
  case b + 1:
    alert("this runs, because +a is 1, exactly equals b+1");
    break;

  default:
    alert("this doesn't run");
}
```

3. `case` ko group karna

Kai `case` ka code **same** ho to unhe group kar sakte ho:

```
javascript

let a = 3;

switch (a) {
  case 4:
    alert('Right!');
    break;

  case 3:    // do cases grouped
  case 5:
    alert('Wrong!');
    alert("Why don't you take a math class?");
    break;

  default:
    alert('The result is strange. Really.');
```

Ab `3` aur `5` dono ke liye same message aayega. Ye `break` na hone ka **side effect** hai: `case 3` se code shuru hokar `case 5` me chala jata hai.

4. Type matter karta hai

Equality check **hamesha** strict hota hai. Value aur type dono same hone chahiye.

```
javascript
```

```

let arg = prompt("Enter a value?");
switch (arg) {
  case '0':
  case '1':
    alert( 'One or zero' );
    break;

  case '2':
    alert( 'Two' );
    break;

  case 3:
    alert( 'Never executes!' );
    break;
  default:
    alert( 'An unknown value' );
}

```

- ①, ② par pehla alert.
- ③ par dusra alert.
- ④ par `default` chalega, kyunki `prompt` string `"3"` deta hai aur `"3" === 3` false hai. Isliye `case 3` **dead code** hai.

Practice Tasks (Answers)

1. `switch` ko `if..else` me badlo:

javascript

```

if (browser == 'Edge') {
  alert("You've got the Edge!");
} else if (browser == 'Chrome'
|| browser == 'Firefox'
|| browser == 'Safari'
|| browser == 'Opera') {
  alert( 'Okay we support these browsers too' );
} else {
  alert( 'We hope that this page looks ok!' );
}

```

(Bilkul same behavior ke liye `===` use karna chahiye, lekin strings ke liye `==` bhi chalta hai. `switch` phir bhi zyada saaf lagta hai.)

2. `if` ko `switch` me badlo:

javascript

```
let a = +prompt('a?', '');

switch (a) {
  case 0:
    alert( 0 );
    break;

  case 1:
    alert( 1 );
    break;

  case 2:
  case 3:
    alert( '2,3' );
    break;
}
```

Aakhri `break` zaruri nahi, lekin future me naya `case` add ho to galti na ho, isliye laga dena achha hai.

Quick Summary

Baat	Yaad rakho
Comparison	Strict (<code>===</code>), type bhi same hona chahiye
<code>break</code>	Na ho to agla <code>case</code> bhi chalta hai (fall-through)
<code>default</code>	Koi <code>case</code> match na ho tab chalta hai (optional)
Grouping	Kai <code>case</code> ek ke neeche ek likho, beech me <code>break</code> mat lagao
Expressions	<code>switch</code> aur <code>case</code> dono me chalte hain